

EcoFlow

Plataforma de Gestión de Energías Renovables

Documento de Arquitectura

Proyecto Final Integrador: El Arquitecto IA

Autor	Robinson Núñez
Institución	Escuela Colombiana de Ingeniería Julio Garavito
Stack	React 19 + TypeScript + Vite + Tailwind CSS 4 + Supabase (PostgreSQL, Auth, RLS)
Fecha	Octubre de 2026
Repositorio	[URL del repositorio: pendiente de agregar]
Video demo	https://youtu.be/ErfbVUhtFNQ

Contenido

- 1. Resumen y cumplimiento de la rúbrica
- 2. Especificación técnica (prompt maestro)
- 3. Diseño de la base de datos
- 4. Seguridad: políticas RLS
- 5. Estructura del frontend
- 6. Implementación en React
- 7. Pruebas y evidencias
- 8. Uso estratégico de la IA
- 9. Limitaciones y trabajo futuro
- 10. Cómo reproducir el proyecto
- Anexo A: esquema SQL completo

1. Resumen y cumplimiento de la rúbrica

EcoFlow es una aplicación interna para que una empresa de tecnología limpia gestione proyectos de instalación de paneles solares. Los administradores crean proyectos y los asignan; cada instalador ve únicamente los suyos y actualiza su estado (*Pendiente, En Progreso, Completado*). El frontend es una SPA en React y TypeScript; el backend es Supabase (PostgreSQL, autenticación y seguridad a nivel de fila).

La solución parte de cuatro decisiones de diseño:

- **Especificación primero.** Las reglas de negocio se escribieron antes que el código (sección 2) y sirvieron como contrato contra el cual se verificó cada resultado de la IA.
- **La seguridad vive en la base de datos.** El filtrado por instalador lo hacen las políticas RLS, no el frontend, de modo que no se puede evadir consultando la API por otro camino.
- **La interfaz nunca toca los datos.** Componentes, hooks y servicios están separados por capas; solo los servicios usan el cliente de Supabase.
- **La IA propone, yo verifico.** Cada salida se contrastó con el compilador, la base de datos real y pruebas con dos usuarios (sección 8).

Cumplimiento de la rúbrica

Criterio	Qué se entrega	Dónde
Precisión en la arquitectura	Esquema con claves foráneas (set null y cascade), check, índices, RLS en las 3 tablas, privilegios por columna y rol protegido, con justificación de cada decisión.	Secciones 3 y 4, Anexo A
Calidad del desarrollo React	Capas modulares, hooks propios, estados de carga y error, validación con zod fuera de los componentes, tipado completo y compilación sin errores.	Secciones 5 y 6
Uso estratégico de la IA	Especificación y prompts, tabla de qué propuso la IA y qué verifiqué, y un problema técnico complejo documentado paso a paso.	Secciones 2 y 8
Entregables	Código fuente, este documento y video demo de 3 a 5 minutos.	Portada

2. Especificación técnica (prompt maestro)

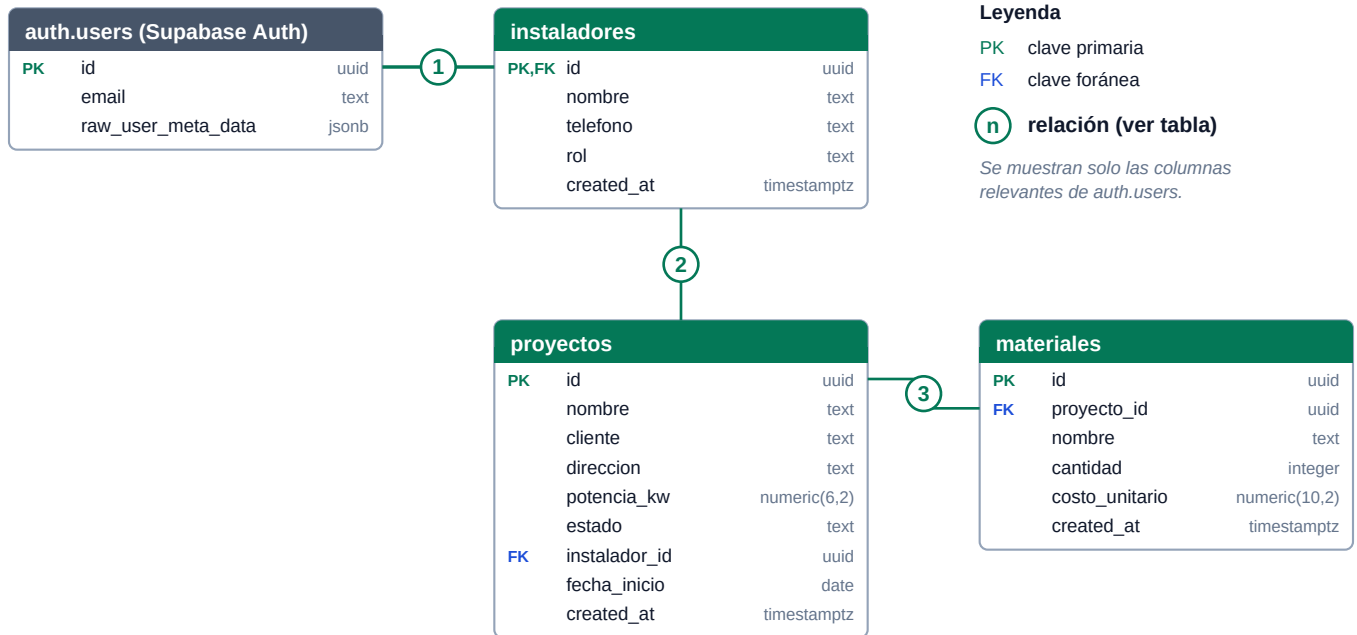
Este documento describe la lógica de negocio con la precisión necesaria para guiar a un asistente de IA y, a la vez, para evaluar sus resultados: cada regla tiene un identificador (RN, SEG, ARQ, CA) que se puede comprobar en el código o en la base de datos. Se redactó antes de escribir el esquema.

Bloque	Contenido
Rol y objetivo	Actúa como arquitecto de software senior en React + TypeScript con Supabase como BaaS. Diseña e implementa EcoFlow: gestión de proyectos de instalación de paneles solares, con esquema relacional correcto, seguridad por fila y un frontend modular y tipado.
Entidades	instaladores: perfil 1 a 1 con <code>auth.users</code> ; nombre, teléfono, rol (instalador o admin). proyectos: nombre, cliente, dirección, potencia en kW, estado, instalador asignado (opcional) y fecha de inicio. materiales: pertenecen a un proyecto; nombre, cantidad y costo unitario.
Reglas de negocio	RN-01 Potencia mayor que 0 y máximo 9999.99 kW; nombre, cliente y dirección no vacíos. RN-02 Estados válidos: Pendiente (por defecto), En Progreso, Completado. RN-03 Solo un administrador crea y elimina proyectos. RN-04 Un instalador solo ve los proyectos asignados a él; el administrador ve todos. RN-05 Un instalador solo puede cambiar el estado de un proyecto, ningún otro dato. RN-06 Al registrarse, el usuario recibe un perfil con rol instalador. El rol no se puede cambiar desde el cliente; el rol admin se asigna en el servidor. RN-07 Al eliminar un proyecto se eliminan sus materiales; los materiales heredan los permisos del proyecto. RN-08 Al eliminar un instalador, sus proyectos no se borran: quedan sin asignar.
Seguridad	SEG-01 RLS activo en las tres tablas, políticas solo para el rol <code>authenticated</code> . SEG-02 Función <code>is_admin()</code> con <code>security definer</code> y <code>search_path</code> vacío, para evitar recursión en las políticas. SEG-03 Privilegios por columna: el cliente solo puede actualizar <code>proyectos.estado</code> e <code>instaladores.nombre/telefono</code> . SEG-04 Mensajes de autenticación genéricos (sin enumeración de usuarios).
Arquitectura	ARQ-01 Capas: <code>components/pages</code> → <code>hooks</code> → <code>services</code> → cliente único de Supabase. ARQ-02 Los componentes nunca llaman a Supabase directamente. ARQ-03 Tipos TypeScript derivados del esquema; la validación de formularios vive fuera de los componentes. ARQ-04 Toda carga de datos expone estados <code>loading</code> y <code>error</code> . ARQ-05 Nombres en inglés para carpetas técnicas; hooks con prefijo <code>use</code> ; servicios con sufijo <code>.service.ts</code> .
Criterios de aceptación	CA-01 El dashboard agrupa los proyectos por estado, con contadores. CA-02 El formulario valida antes de enviar y muestra un mensaje por campo. CA-03 Un instalador con un proyecto asignado ve solo ese proyecto y no ve el formulario de creación. CA-04 El administrador ve todos los proyectos y puede crear nuevos. CA-05 El proyecto compila sin errores de TypeScript (<code>tsc -b</code>).
Formato de salida	Entregar por fases verificables; en cada fase indicar cómo comprobar el resultado antes de continuar.

Cómo se usó. Esta especificación fue la referencia para revisar cada entrega de la IA: por ejemplo, CA-03 es exactamente lo que demuestra la prueba de RLS de la sección 7, y SEG-03 explica por qué un administrador tampoco puede editar otras columnas desde el cliente (limitación documentada en la sección 9).

3. Diseño de la base de datos

El modelo tiene tres tablas propias, enlazadas con la tabla de autenticación de Supabase. El esquema completo está en el Anexo A y se aplicó de forma atómica (todo o nada).



#	Relación	Cardinalidad	Regla al borrar	Justificación
1	auth.users -> instaladores	1 a 1	on delete cascade	Un perfil no tiene sentido sin su usuario de autenticación.
2	instaladores -> proyectos	1 a N	on delete set null	Si se elimina un instalador no se pierde el historial de proyectos; quedan sin asignar (RN-08).
3	proyectos -> materiales	1 a N	on delete cascade	Un material pertenece a un único proyecto y no existe sin él (RN-07).

Integridad de datos en el servidor

Las restricciones viven en la base de datos, no solo en el formulario, de modo que ningún cliente puede saltárselas:

- check (potencia_kw > 0) con tipo numeric(6,2) (máximo 9999.99).
- check (estado in ('Pendiente', 'En Progreso', 'Completado')) con valor por defecto 'Pendiente'.
- check (length(trim(nombre)) > 0) en nombre y cliente; cantidad > 0 y costo_unitario >= 0 en materiales.
- check (rol in ('instalador', 'admin')).
- Índices en las columnas de consulta frecuente: proyectos(instalador_id), proyectos(estado) y materiales(proyecto_id). El primero además acelera la evaluación de RLS.

- Trigger on_auth_user_created: al registrarse un usuario crea su fila en instaladores con rol instalador. El rol no viaja desde el cliente.

4. Seguridad: políticas RLS

Las tres tablas tienen RLS activado y todas las políticas se aplican al rol `authenticated`. Sin política, la operación se deniega por defecto.

Tabla	Política	Operación	Condición	Por qué
instaladores	instaladores_select	SELECT	<code>auth.uid() = id</code> o <code>is_admin()</code>	Cada usuario lee su perfil; el admin necesita ver a todos para asignar proyectos.
instaladores	instaladores_update_own	UPDATE	solo su propia fila	Cada quien edita su perfil, y solo nombre y teléfono (privilegio por columna).
proyectos	proyectos_select	SELECT	<code>instalador_id = auth.uid()</code> o <code>is_admin()</code>	Cumple RN-04: el instalador solo ve lo asignado.
proyectos	proyectos_insert_admin	INSERT	<code>is_admin()</code>	RN-03.
proyectos	proyectos_update	UPDATE	asignado a él o admin (<i>using</i> y <i>with check</i>)	El instalador actualiza el estado de lo suyo; <i>with check</i> impide reasignarlo a otro.
proyectos	proyectos_delete_admin	DELETE	<code>is_admin()</code>	RN-03.
materiales	materiales_select	SELECT	exists del proyecto visible para el usuario	Los materiales heredan el permiso del proyecto (RN-07).
materiales	materiales_insert	INSERT	ídem, sobre el proyecto padre	Solo se agregan materiales a proyectos accesibles.

Decisiones clave

Función `is_admin()` con `security definer`. Una política de instaladores que consultara la propia tabla `instaladores` para saber si el usuario es admin se evaluaría a sí misma y provocaría recursión. La función se ejecuta con los permisos de su dueño, evita el ciclo y usa `search_path` vacío para que no se pueda suplantar una tabla con un esquema falso.

`(select auth.uid())` **en vez de** `auth.uid()`. Envolver la llamada en un `select` permite que Postgres la evalúe una vez por consulta y no una vez por fila, lo que mejora el rendimiento al crecer la tabla.

Privilegios por columna. Además de RLS (que decide *qué filas*), se revocó `update` a `authenticated` y se concedió solo sobre `proyectos(estados)` e `instaladores(nombre, telefono)`. Así RLS limita las filas y los privilegios limitan las columnas: un instalador no puede cambiar el cliente, la potencia ni, sobre todo, su propio rol. Esto cierra el ataque típico de escalada de privilegios desde el cliente.

proyectos API DISABLED			Disable RLS	Create policy	
This table has custom Data API permissions — access may be restricted for some roles or operations.					
NAME	COMMAND	APPLIED TO			
proyectos_delete_admin	DELETE	authenticated			
proyectos_insert_admin	INSERT	authenticated			
proyectos_select	SELECT	authenticated			
proyectos_update	UPDATE	authenticated			

Figura 1. Consola de Supabase: políticas RLS de la tabla *proyectos* (SELECT, INSERT, UPDATE y DELETE, aplicadas a *authenticated*).

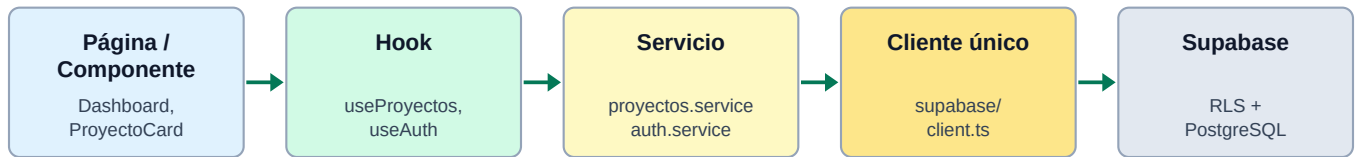
instaladores API DISABLED			Disable RLS	Create policy	
This table has custom Data API permissions — access may be restricted for some roles or operations.					
NAME	COMMAND	APPLIED TO			
instaladores_select	SELECT	authenticated			
instaladores_update_own	UPDATE	authenticated			

materiales API DISABLED			Disable RLS	Create policy	
This table has custom Data API permissions — access may be restricted for some roles or operations.					
NAME	COMMAND	APPLIED TO			
materiales_insert	INSERT	authenticated			
materiales_select	SELECT	authenticated			

Figura 2. Políticas de *instaladores* (SELECT, UPDATE) y *materiales* (INSERT, SELECT). El botón *Disable RLS* indica que RLS está activo.

5. Estructura del frontend

El equipo anterior mezclaba lógica de negocio con la interfaz, y eso producía errores recurrentes. La estructura obliga a separar cada responsabilidad en su capa.



Las componentes nunca importan el cliente de Supabase: solo llaman a hooks.

```
src/
|-- components/
|   |-- layout/           ProtectedRoute.tsx
|   |-- proyectos/       EstadoBadge.tsx  ProyectoCard.tsx  ProyectoForm.tsx
|   |-- ui/
|-- context/             AuthContext.ts  AuthProvider.tsx
|-- hooks/               useAuth.ts   useInstaladores.ts  useProyectos.ts
|-- pages/               Dashboard.tsx  Login.tsx  Register.tsx  Home.tsx ...
|-- services/
|   |-- supabase/        client.ts          <- cliente unico de Supabase
|   |-- auth.service.ts
|   |-- instaladores.service.ts
|   |-- proyectos.service.ts
|-- types/               database.types.ts  proyecto.schema.ts
|-- utils/               estados.ts
```

Carpeta	Responsabilidad	Qué NO debe contener
components/	Piezas de interfaz reutilizables. Reciben datos y callbacks por props.	Llamadas a Supabase, reglas de validación, lógica de negocio.
pages/	Una pantalla por ruta: compone componentes y usa hooks.	Consultas a la base de datos; fetch directo.
hooks/	Estado de carga, error y datos; orquestan los servicios.	JSX, estilos, acceso directo al cliente de Supabase.
context/	Estado global compartido: sesión y perfil del usuario.	Datos de dominio que pertenecen a un solo hook.
services/	Único lugar que consulta tablas y autenticación, una función por operación.	Estado de React, JSX, mensajes de interfaz.
services/supabase /	Cliente único y su configuración (URL y clave anónima desde variables de entorno).	Consultas por entidad (van en *.service.ts).
types/	Contratos de datos: tipos del esquema y esquemas de validación.	Componentes ni acceso a datos.
utils/	Constantes y funciones puras sin dependencias de React ni Supabase.	Estado, efectos, llamadas de red.

Escalabilidad

Agregar un módulo nuevo (por ejemplo, *materiales* o *reportes*) implica crear un servicio, un hook y sus componentes, sin tocar lo existente. Los tipos de la tabla ya están definidos en `database.types.ts`.

6. Implementación en React

Flujo de datos

Dashboard -> useProyectos -> proyectos.service.ts -> client.ts -> Supabase. La página solo conoce el hook, y el hook solo conoce el servicio.

Hooks y estados de carga y error

- useProyectos devuelve { proyectos, loading, error, recargar, crear, cambiarEstado }. Cancela la actualización de estado si el componente se desmonta antes de que responda la red.
- AuthProvider expone la sesión, el perfil y esAdmin; mientras la sesión o el perfil cargan, loading es verdadero y ProtectedRoute muestra «Cargando...» en vez de redirigir por error.
- El dashboard muestra tres estados explícitos: cargando, error con botón *Reintentar* y lista vacía con un mensaje distinto para el administrador y para el instalador.
- Las acciones que escriben (crear, cambiarEstado) capturan el error del servicio y lo muestran junto al control que lo originó.

Tipado

Los tipos de Database reflejan el esquema, incluidas las uniones de texto (EstadoProyecto, RolInstalador). El tipo Update de *proyectos* solo admite estado, igual que el privilegio por columna del SQL, de modo que el compilador impide intentar editar otra columna. El cliente se crea como createClient<Database>, por lo que toda consulta queda tipada. El proyecto compila con tsc -b y npm run build sin errores.

Validación antes de enviar a Supabase

La validación está en types/proyecto.schema.ts, fuera del componente. ProyectoForm solo llama a validarProyecto(); si hay errores los muestra por campo y no se envía nada a la base de datos.

```
export const proyectoSchema = z.object({
  nombre: z.string().trim().min(3, 'El nombre debe tener al menos 3 caracteres.').max(80, ...),
  cliente: z.string().trim().min(2, ...).max(80, ...),
  direccion: z.string().trim().min(5, ...).max(150, ...),
  potencia_kw: z.coerce
    .number({ error: 'Ingresa un número válido.' })
    .positive('La potencia debe ser mayor que 0.')
    .max(9999.99, 'La potencia máxima es 9999.99 kW.'),
  instalador_id: z.string(),
  fecha_inicio: z.union([z.literal(''), z.iso.date('Fecha no válida.')] ),
})
```

Hay doble validación a propósito: zod da respuesta inmediata al usuario y los check de PostgreSQL garantizan la regla aunque otro cliente llegue a la API.

Interfaz con Tailwind CSS

La interfaz usa Tailwind CSS 4: tres columnas por estado con contador, insignias de color por estado, formulario en rejilla responsiva, y mensajes accesibles (role="alert" en errores, aria-busy en cargas).

7. Pruebas y evidencias

Prueba de RLS con dos usuarios

Se creó un segundo usuario con rol instalador (*perseidasr*) y se asignaron a él dos proyectos. Un tercer proyecto se asignó al administrador. El administrador ve los tres proyectos (Figura 3) y el instalador ve únicamente los dos suyos (Figura 4), sin botón *Nuevo proyecto* ni formulario.

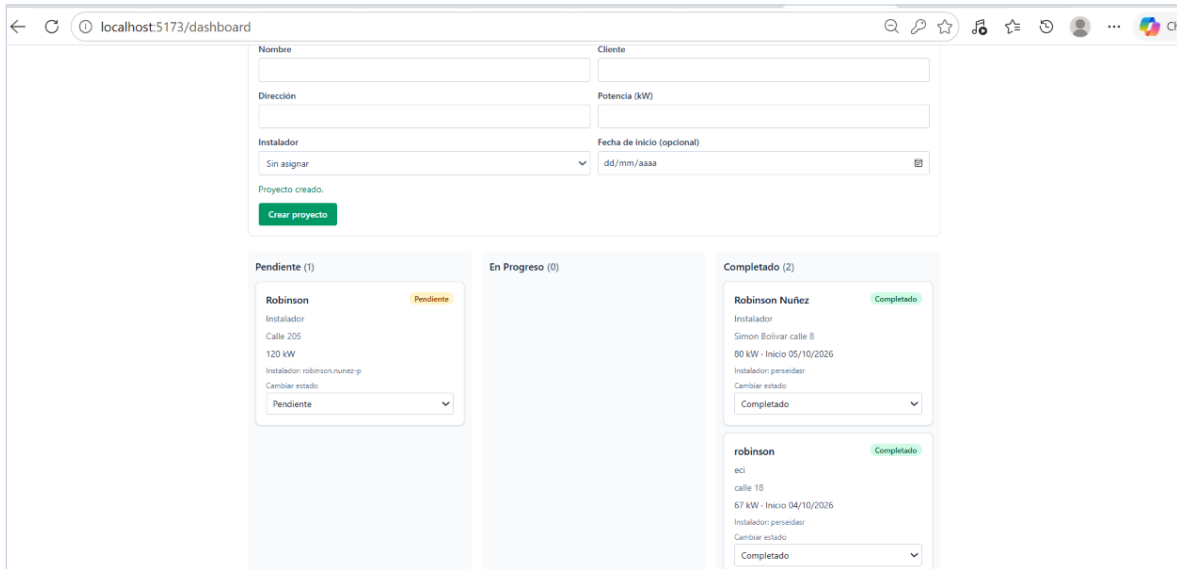


Figura 3. Vista del administrador: 3 proyectos (1 Pendiente y 2 Completado), formulario de creación y mensaje «Proyecto creado.»

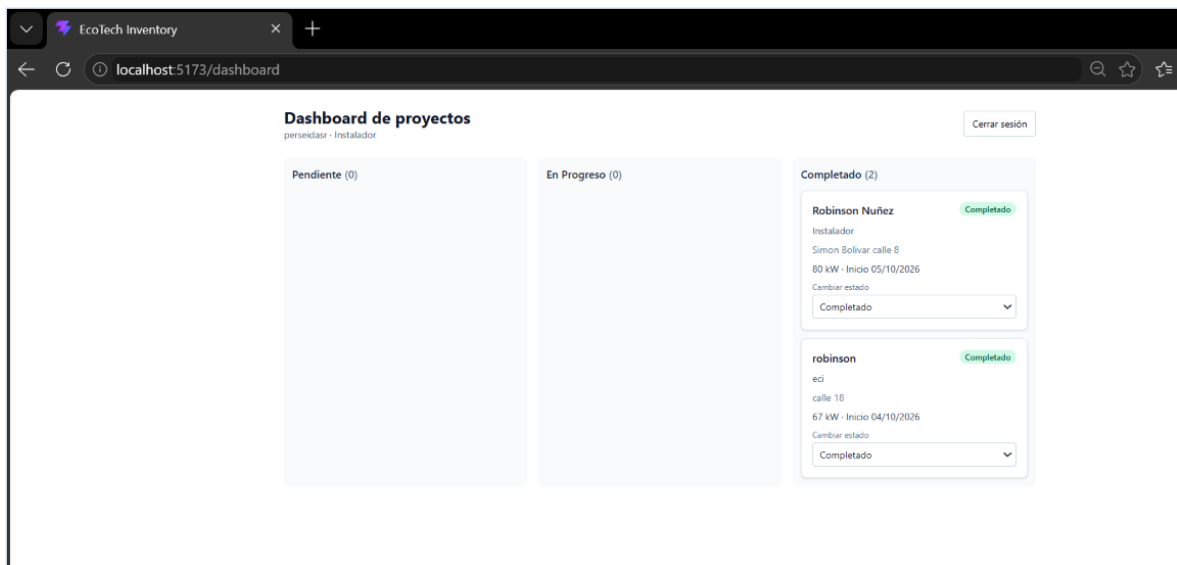


Figura 4. Vista del instalador *perseidasr*: solo ve los 2 proyectos asignados a él. El filtrado lo hace la política *proyectos_select* en la base de datos.

Una corrección sobre la marcha. La primera prueba no demostraba nada: ambos usuarios veían los mismos dos proyectos porque estaban asignados al mismo instalador. Se agregó un tercer proyecto asignado a otro usuario para que la diferencia (3 frente a 2) probara el aislamiento.

Pruebas de la validación (8 casos)

Se ejecutó `validarProyecto()` con entradas válidas e inválidas:

#	Entrada	Resultado
1	Datos válidos, potencia "5.5"	Aceptado; potencia convertida a número 5.5; instalador y fecha vacíos pasan a <i>null</i>
2	Nombre, cliente y dirección vacíos	Rechazado; un mensaje por cada campo
3	Potencia "0"	Rechazado: «La potencia debe ser mayor que 0.»
4	Potencia "abc"	Rechazado: «Ingresa un número válido.»
5	Potencia vacía	Rechazado: «La potencia debe ser mayor que 0.»
6	Potencia "99999"	Rechazado: «La potencia máxima es 9999.99 kW.»
7	Fecha "31/12/2026"	Rechazado: «Fecha no válida.»
8	Fecha "2026-11-01" con instalador	Aceptado con ambos valores

Resumen de verificaciones

Verificación	Resultado	Evidencia
Compilación TypeScript (<code>tsc -b</code>) y <code>npm run build</code>	Sin errores	Salida de la terminal
Aplicación atómica del esquema SQL	OK: esquema aplicado	Terminal y Table Editor
Creación de proyecto desde el formulario	Funciona	Figura 3
Cambio de estado (columnas Pendiente / Completado)	Funciona	Figuras 3 y 4
Admin ve todos; instalador solo los suyos (CA-03 y CA-04)	Cumple	Figuras 3 y 4
Políticas RLS activas en las 3 tablas	Cumple	Figuras 1 y 2

8. Uso estratégico de la IA

Método de trabajo

- **Especificar antes de pedir.** Cada fase parte de una especificación (sección 2) y de un criterio de verificación que se comprueba antes de continuar.
- **Fases pequeñas y verificables.** El proyecto se dividió en cinco fases; cada una termina con algo que se ejecuta y se comprueba (SQL aplicado, compilación, pantalla funcionando).
- **Contrastar con la realidad.** Los resultados se verificaron con el compilador, la base de datos real y dos usuarios, no se aceptaron por verse correctos.
- **Yo decido el diseño.** Elegí el stack, el modelo de datos, el rol protegido y el alcance; la IA genera borradores que reviso.

Prompts clave

Prompt 1: estructura de carpetas con Desarrollo Basado en Especificaciones (SDD). Definí rol, contexto, restricciones y formato de salida antes de escribir código:

Actúa como un Arquitecto de Software Senior especializado en aplicaciones React + TypeScript con Supabase como backend (BaaS).

CONTEXTO: Trabajo en "EcoTech Solutions", una startup que desarrolla una aplicación de gestión de inventario sostenible. Stack: React 18 + Vite + TypeScript + Supabase. El equipo es junior y suele mezclar la lógica de negocio con los componentes de UI.

TAREA: Diseña la estructura de carpetas ideal dentro de /src para el scaffolding inicial, aplicando Desarrollo Basado en Especificaciones (SDD).

RESTRICCIONES TÉCNICAS

1. Separación estricta: UI (components, pages), hooks, estado global (context), acceso a datos (services) y contratos de datos (types).
2. Los componentes NUNCA llaman a Supabase directamente: solo a través de hooks que consuman la capa de servicios.
3. Carpeta específica para Supabase (cliente único), separada de los servicios por entidad.
4. Escalable: agregar módulos (productos, proveedores, reportes) sin reorganizar.
5. Nombres en inglés; componentes PascalCase, hooks con prefijo "use", servicios ".service.ts".
6. Sin sobreingeniería: apropiada para un equipo pequeño y junior.

FORMATO DE SALIDA: árbol de directorios, tabla (carpeta | responsabilidad | qué NO debe contener), comandos mkdir para PowerShell y Bash, y una frase por carpeta sobre Supabase.

Esa estructura es la base de la sección 5. Los demás prompts del proyecto se resumen en la tabla siguiente, junto con lo que verifiqué o corregí en cada caso:

Solicitud a la IA	Qué propuso	Qué verifiqué o decidí
Pegar el enunciado y pedir un plan por fases con entregable verificable en cada una	Cinco fases: esquema y RLS, tipos y hooks, dashboard y formulario, problema con IA, documento y video.	Aprobé el orden y ejecuté cada fase antes de pasar a la siguiente.
Esquema SQL y RLS para instaladores, proyectos y materiales	Tablas, check, índices, is_admin(), políticas, privilegios por columna y trigger.	Conservé los 3 puntos a defender (claves foráneas, RLS, rol protegido) y comprobé el efecto con dos usuarios.
Tipos TypeScript, cliente, servicio y hook useProyectos (le pasé el árbol real de mi proyecto)	database.types.ts, client.ts, proyectos.service.ts, useProyectos.ts.	Verifiqué los tipos contra el esquema y ejecuté tsc -b y npm run build.

Solicitud a la IA	Qué propuso	Qué verifiqué o decidí
Dashboard, formulario validado y login	Páginas, componentes, esquema zod, AuthProvider y rutas protegidas.	Descubrí que faltaba un login (sin sesión, RLS no devuelve datos) y lo incorporé; probé el flujo completo.
Revisar el proyecto base que subí en un .zip	Detectó zod sin declarar en package.json, un console.log que imprimía las variables de entorno y que .env no estaba en .gitignore.	Corregí los tres puntos antes de subir el repositorio.

Problema técnico complejo resuelto con IA: RLS sin recursión

El problema. Había que garantizar que cada instalador viera solo sus proyectos, y que el administrador pudiera ver todo. La regla natural para «¿es admin?» consulta la tabla `instaladores`, pero esa tabla también tiene RLS: una política de `instaladores` que consulta `instaladores` se evalúa a sí misma y Postgres falla por recursión infinita.

Paso	Detalle
1. Enfoque	Pedí aislar la comprobación de rol en un único punto reutilizable por todas las políticas, sin duplicar la lógica.
2. Propuesta de la IA	Función <code>public.is_admin()</code> con <code>security definer</code> y <code>search_path = ''</code> , y políticas que la invocan junto con <code>(select auth.uid())</code> .
3. Revisión del diseño	Entendí por qué funciona: <code>security definer</code> ejecuta la consulta con los permisos del dueño y evita el ciclo; el <code>search_path</code> vacío impide suplantar objetos; el <code>select</code> alrededor de <code>auth.uid()</code> evita reevaluarla por fila.
4. Segunda capa	Revoqué <code>update</code> general y concedí solo <code>update (estado)</code> en proyectos para que un instalador no pueda editar otras columnas ni su propio rol.
5. Verificación	Prueba con dos usuarios: el administrador ve 3 proyectos y el instalador solo los 2 asignados, sin el formulario de creación (Figuras 3 y 4).

Otros incidentes resueltos durante el desarrollo

Incidente	Diagnóstico	Solución y verificación
El SQL Editor de Supabase se quedaba cargando	Se sospechó de la red (carga de componentes del editor) y no del SQL. El esquema no dependía del editor.	Script de Node con pg conectado por el <i>Session pooler</i> y envuelto en <code>begin/commit/rollback</code> , para que el esquema se aplique completo o no se aplique. Verificado con «OK» y el Table Editor.
<code>syntax error at or near "update"</code> con un SQL correcto	El archivo creado con Set-Content -Encoding utf8 en Windows PowerShell 5.1 empieza con una marca BOM invisible (EF BB BF) que Postgres lee como parte del primer token.	El script elimina la marca con <code>.replace(/\^uFEFF/, "")</code> . Verificado: el <code>update</code> corrió y el dashboard pasó a mostrar «Administrador».
Un <code>update</code> respondía OK, pero el usuario seguía siendo instalador	Un <code>update</code> que no encuentra filas termina con éxito: el correo del filtro no coincidía con el del usuario.	Se filtró por el prefijo del correo con <code>like</code> y se confirmó el resultado en la tabla. Lección: verificar el estado, no el mensaje del comando.

9. Limitaciones y trabajo futuro

Estas son las limitaciones conocidas de la entrega y cómo se resolverían:

Limitación	Por qué	Siguiente paso
La interfaz no gestiona <i>materiales</i>	El alcance de esta entrega es el dashboard y el formulario de proyectos. La tabla, sus tipos y sus políticas RLS ya existen.	Crear <code>materiales.service.ts</code> , un hook y sus componentes.
Nadie puede editar el cliente ni la potencia de un proyecto desde la aplicación	Por SEG-03 se revocó update general, y eso aplica también al administrador.	Una función RPC <code>security.definer</code> que valide que el llamador es admin.
<i>materiales</i> no tiene políticas de UPDATE ni DELETE	Solo se definió lectura e inserción; el resto se deniega por defecto.	Agregar ambas políticas heredando el permiso del proyecto, cuando exista la interfaz.
<code>Register.tsx</code> llama a Supabase directamente	Es una página heredada del proyecto base que no cumple ARQ-02.	Mover <code>signUp</code> a <code>auth.service.ts</code> y usar <code>useAuth</code> .
Los tipos se escribieron a mano desde el esquema	La CLI de Supabase depende de la red y no se pudo usar de forma fiable.	Generarlos con <code>supabase gen types typescript</code> y compararlos con los actuales.
No hay pruebas automatizadas en el repositorio	La validación se probó con un script puntual y el resto manualmente.	Añadir Vitest para <code>validarProyecto</code> y pruebas de integración de RLS.
Sin paginación en la lista de proyectos	Con pocos proyectos no es necesaria.	Paginar con <code>range()</code> de Supabase.

10. Cómo reproducir el proyecto

1. Clonar e instalar dependencias

```
git clone <URL_DEL_REPOSITORIO>
cd ecotech-inventory
npm install
```

2. Variables de entorno. Copiar `.env.example` a `.env` y completar los valores de su proyecto de Supabase (Project Settings, API). El archivo `.env` está en `.gitignore` y no se sube al repositorio.

```
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
```

3. Aplicar el esquema con la cadena del *Session pooler* (Connect, Connection string). En PowerShell:

```
npm install -D pg
$env:DATABASE_URL = "postgresql://postgres.<REF>:<CLAVE>@aws-0-<REGION>.pooler.supabase.com:5432/postgres"
node apply-sql.mjs schema.sql
```

Debe responder OK: `schema.sql` aplicado correctamente. El script es atómico.

4. Crear un administrador. Registrar un usuario (Authentication, Users, Add user) y ejecutar con `node apply-sql.mjs admin.sql`:

```
update public.instaladores set rol = 'admin'
where id = (select id from auth.users where email = 'CORREO_DEL_ADMIN');
```

5. Iniciar la aplicación

```
npm run dev      # http://localhost:5173/login
npm run build    # compilación de producción (incluye tsc -b)
```

Anexo A: esquema SQL completo

Archivo `schema.sql`, aplicado de forma atómica.

```
-- EcoFlow: esquema, RLS y trigger
-- Se aplica de forma atómica: si algo falla, no se crea nada.

-- 1. Tablas
create table public.instaladores (
  id          uuid primary key references auth.users (id) on delete cascade,
  nombre      text not null,
  telefono    text,
  rol         text not null default 'instalador'
              check (rol in ('instalador', 'admin')),
  created_at  timestampz not null default now()
);

create table public.proyectos (
  id          uuid primary key default gen_random_uuid(),
  nombre      text not null check (length(trim(nombre)) > 0),
  cliente     text not null check (length(trim(cliente)) > 0),
  direccion   text not null,
  potencia_kw numeric(6,2) not null check (potencia_kw > 0),
  estado      text not null default 'Pendiente'
              check (estado in ('Pendiente', 'En Progreso', 'Completado')),
  instalador_id uuid references public.instaladores (id) on delete set null,
  fecha_inicio date,
  created_at  timestampz not null default now()
);

create table public.materiales (
  id          uuid primary key default gen_random_uuid(),
  proyecto_id uuid not null references public.proyectos (id) on delete cascade,
  nombre      text not null check (length(trim(nombre)) > 0),
  cantidad    integer not null check (cantidad > 0),
  costo_unitario numeric(10,2) not null default 0 check (costo_unitario >= 0),
  created_at  timestampz not null default now()
);

-- 2. Índices
create index idx_proyectos_instalador on public.proyectos (instalador_id);
create index idx_proyectos_estado    on public.proyectos (estado);
create index idx_materiales_proyecto on public.materiales (proyecto_id);

-- 3. Función auxiliar (evita recursión en RLS)
create or replace function public.is_admin()
returns boolean
language sql
security definer
set search_path = ''
stable
as $$
  select exists (
    select 1 from public.instaladores
    where id = (select auth.uid()) and rol = 'admin'
  );
$$;

-- 4. RLS
alter table public.instaladores enable row level security;
alter table public.proyectos   enable row level security;
alter table public.materiales  enable row level security;

-- instaladores: cada uno ve y edita lo suyo; el admin ve todos
```

```

create policy "instaladores_select" on public.instaladores for select
    to authenticated using ((select auth.uid() = id or public.is_admin());
create policy "instaladores_update_own" on public.instaladores for update
    to authenticated using ((select auth.uid() = id)
    with check ((select auth.uid() = id);

-- proyectos: el instalador solo ve los asignados a él
create policy "proyectos_select" on public.proyectos for select
    to authenticated
    using (instalador_id = (select auth.uid()) or public.is_admin());
create policy "proyectos_insert_admin" on public.proyectos for insert
    to authenticated with check (public.is_admin());
create policy "proyectos_update" on public.proyectos for update
    to authenticated
    using (instalador_id = (select auth.uid()) or public.is_admin())
    with check (instalador_id = (select auth.uid()) or public.is_admin());
create policy "proyectos_delete_admin" on public.proyectos for delete
    to authenticated using (public.is_admin());

-- materiales: se heredan los permisos del proyecto
create policy "materiales_select" on public.materiales for select
    to authenticated using (exists (
        select 1 from public.proyectos p
        where p.id = materiales.proyecto_id
        and (p.instalador_id = (select auth.uid()) or public.is_admin())));
create policy "materiales_insert" on public.materiales for insert
    to authenticated with check (exists (
        select 1 from public.proyectos p
        where p.id = materiales.proyecto_id
        and (p.instalador_id = (select auth.uid()) or public.is_admin())));

-- 5. Privilegios por columna: un instalador solo cambia el estado de un proyecto
revoke update on public.proyectos from authenticated;
grant update (estado) on public.proyectos to authenticated;
revoke update on public.instaladores from authenticated;
grant update (nombre, telefono) on public.instaladores to authenticated;

-- 6. Crear el instalador al registrarse
create or replace function public.handle_new_user()
returns trigger language plpgsql security definer set search_path = ''
as $$
begin
    insert into public.instaladores (id, nombre)
    values (new.id, coalesce(new.raw_user_meta_data ->> 'name', split_part(new.email, '@', 1)));
    return new;
end;
$$;

create trigger on_auth_user_created
after insert on auth.users
for each row execute function public.handle_new_user();

```